

Sales Forecasting Project Documentation

Introduction

Retail businesses operate in a highly dynamic environment where sales performance is influenced by promotions, holidays, economic conditions, and customer behavior. Accurately forecasting sales enables companies to optimize inventory, staffing, and operational planning. The purpose of this project is to build a robust forecasting system capable of predicting daily and weekly sales across more than 1,000 Rossmann stores using the Kaggle Rossmann dataset. This documentation outlines every phase of the project including data cleaning, feature engineering, modeling, evaluation, and future improvements.

Machine Learning Models Overview

Machine learning models are designed to learn patterns from historical data and generate predictions without being explicitly programmed for each scenario. They are ideal for forecasting retail sales because they can capture complex interactions between promotions, store characteristics, competition, and seasonal patterns. This project uses three major models: SARIMAX for classical time-series forecasting, XGBoost for nonlinear pattern learning, and LightGBM for scalable gradient boosting.

Model Descriptions

SARIMAX (Seasonal AutoRegressive Integrated Moving Average with Exogenous Variables):

SARIMAX is a time-series forecasting model that expands ARIMA by including seasonal patterns and external regressors. It captures

temporal dependencies, seasonality, and holiday effects. It performed strongly due to clear yearly trends in Rossmann sales.

XGBoost (Extreme Gradient Boosting):

XGBoost is a powerful ensemble model based on boosted decision trees. It excels at modeling nonlinear relationships. In this project, it used engineered features such as competition duration and promo activity. It performed well but was slower than LightGBM.

LightGBM (Light Gradient Boosting Machine):

LightGBM is a highly optimized gradient boosting framework using leaf-wise growth. It is faster and consumes less memory than XGBoost while often achieving higher accuracy. In this project, LightGBM delivered the second-best performance after SARIMAX, making it ideal for large-scale deployment.

Training Process and Methodology

1. Data Cleaning:

- Parsed dates and indexed the dataset by Date.
- Merged store information into the training dataset.
- Filled missing competition and promo fields using Iterative Imputation.
- Detected outliers but retained them due to business relevance.

2. Feature Engineering:

- Extracted Year, Month, Day, WeekOfYear.

- Created SalesPerCustomer, Promo month indicators, and competition/promo duration.
- Applied log-transforms where needed.

3. Encoding and Scaling:

- One-hot encoded StoreType, Assortment, StateHoliday.
- Scaled numerical features using MinMaxScaler for LightGBM.
- SARIMAX used only Sales values (weekly aggregated).

4. Modeling:

- SARIMAX: searched parameter combinations using AIC.
- XGBoost: trained with engineered features and early stopping.
- LightGBM: trained with regularization and evaluated using RMSE.

5. Evaluation:

RMSE was selected as the main metric. SARIMAX achieved the lowest RMSE due to strong seasonal patterns, followed by LightGBM, and then XGBoost.

MLOps Flow

This project includes deployment and MLOps steps integrated into the training pipeline so models can be tracked, inspected, and served. The following flow documents the implemented steps (scripts and tools).

1) Artifact creation and local persistence:

- - Training scripts save model artifacts as joblib files in the ``models/`` directory. Typical artifact structure for LGBM: a dict with keys ``model``, ``scaler``, and ``feature_columns``. SARIMAX artifacts include a ``results`` object and ``train_end``.

2) MLflow logging & experiment tracking:

- - Using ``--mlflow`` in ``scripts.train`` creates an MLflow run where parameters (hyperparameters), metrics (RMSE) and artifacts (.joblib) are logged.
- - LightGBM training uses MLflow autologging (when available) to capture iteration-level metrics and training lineage automatically.

3) Pyfunc model wrapping for consistent inference:

- - A SARIMAX pyfunc wrapper (``scripts/sarimax_pyfunc.py``) is provided so SARIMAX models are exposed as `mlflow.pyfunc`` models (artifact key ``model_pyfunc``).
- - A LightGBM pyfunc wrapper (``scripts/lgbm_pyfunc.py``) was added so LGBM artifacts can be logged and used through the ``mlflow.pyfunc`` interface; this wrapper aligns features, applies the scaler, and calls the model predict method.

4) Inspecting artifacts:

- - ``scripts/list_mlflow_artifacts.py`` lists artifacts stored in a named MLflow experiment and can be used to audit runs programmatically.

5) Serving and UI integration:

- - The Dash UI (`app.py`) consumes the SARIMAX artifact directly for forecasting. When LGBM is logged with pyfunc, it becomes loadable via `mlflow.pyfunc.load_model()` and suitable for serving (batch or REST).

6) Reproducibility & environment capture:

- - The logging helpers capture auxiliary artifacts (scaler, feature_columns) and the log helper can record `pip freeze` output to help reproduce the training environment.

Deployment

This project was migrated from a single analysis notebook into a reproducible training and deployment pipeline. Below are the concrete steps implemented to enable tracking, packaging, serving and reproducible inference.

1) Notebook → Modular scripts (reproducible codebase):

The exploratory Jupyter notebook was refactored into small, importable Python modules and CLI helpers placed under `scripts/`. Key files: `scripts/data_loader.py` (preprocessing/features), `scripts/train.py` (training for LGBM or SARIMAX + mlflow integration), `scripts/predict.py` (CLI prediction), `scripts/log_lgbmlflow.py` (retroactive MLflow logging helper), `scripts/list_mlflow_artifacts.py` (artifact auditing), `scripts/sarimax_pyfunc.py` and `scripts/lgbm_pyfunc.py` (pyfunc wrappers).

2) Training and reproducible artifacts:

Training produces joblib artifacts in `models/` (e.g., `lgb_artifacts.joblib`, `sarimax_artifacts.joblib`). These artifacts are

structured and include preprocessing metadata (scaler, feature_columns) to enable reproducible inference.

3) MLflow experiment tracking & model registry readiness:

``scripts.train --mlflow`` logs runs to MLflow (params, metrics, artifacts). LightGBM autologging is enabled to capture iteration-level metrics and training lineage. When possible training also logs a ``pyfunc`` model (artifact ``model_pyfunc``) for consistent serving.

The helper ``scripts/log_lgbmlflow.py`` supports retroactive logging of an existing joblib artifact and, with ``--val-data``, computes RMSE and attaches predictions as artifacts. This helps import previously-saved models into MLflow for auditing and serving.

4) Serving / UI integration:

The Dash app (``app.py``) provides a simple prediction UI/endpoint for SARIMAX artifacts. It loads SARIMAX joblib artifacts directly and writes forecasts to ``results/sarimax_ui_predictions.csv``. When LGBM is logged as a pyfunc model, ``mlflow.pyfunc.load_model()`` can be used to load the full preprocessing + model for batch serving or model server deployment.

5) Local deployment flow used during this project:

1) Start local MLflow.

2) Train a model with MLflow enabled using ``scripts.train --mlflow``.

3) Inspect runs in the MLflow UI (``mlruns/`` or server).

4) Export model as pyfunc or use ``scripts/log_lgbmlflow.py`` to log existing artifacts.

5) Use ``mlflow.pyfunc.load_model()`` or ``mlflow models serve`` for alternate serving.

Conclusion

This project successfully developed a forecasting system for Rossmann sales using classical and modern machine learning methods. The SARIMAX model performed best because it effectively captured yearly and monthly seasonality. LightGBM provided excellent performance at scale, making it suitable for real-time systems. XGBoost performed well but lagged slightly behind LightGBM. Together, these models provide valuable insight into sales behavior and highlight the benefits of combining time-series and machine-learning approaches.

Room for Improvement

Several enhancements could further improve forecasting accuracy:

1. Deep Learning Models:
 - a. LSTM, GRU, and Temporal Convolutional Networks can capture long-term dependencies.
2. Additional Feature Engineering:
 - a. Incorporate lag features, rolling windows, weather data, macroeconomic indicators, and geospatial factors.
3. Improved Validation:

Implement walk-forward validation for better temporal evaluation.
4. Per-Store Modeling:

Train separate models for each store or store type for more localized accuracy.
5. Add CI-left automation to evaluate candidate runs:

automatically compute evaluation metrics and register best models to the Model Registry.

6. Define a promotion pipeline: automated tests (unit + integration + performance) that run in CI, then use MlflowClient to transition model versions (Staging → Production).
7. Containerize the Dash app and an MLflow server + model server. Provide Kubernetes / Docker Compose manifest to deploy the entire stack and ensure reproducible serving environments.
8. Integrate monitoring and model drift detection and set up automated retraining triggers (e.g., when RMSE degrades beyond a threshold).

References

Kaggle Rossmann Dataset: <https://www.kaggle.com/c/rossmann-store-sales>

Statsmodels SARIMAX Documentation: <https://www.statsmodels.org/>

XGBoost Documentation: <https://xgboost.readthedocs.io/>

LightGBM Documentation: <https://lightgbm.readthedocs.io/>

Scikit-learn Documentation: <https://scikit-learn.org/>